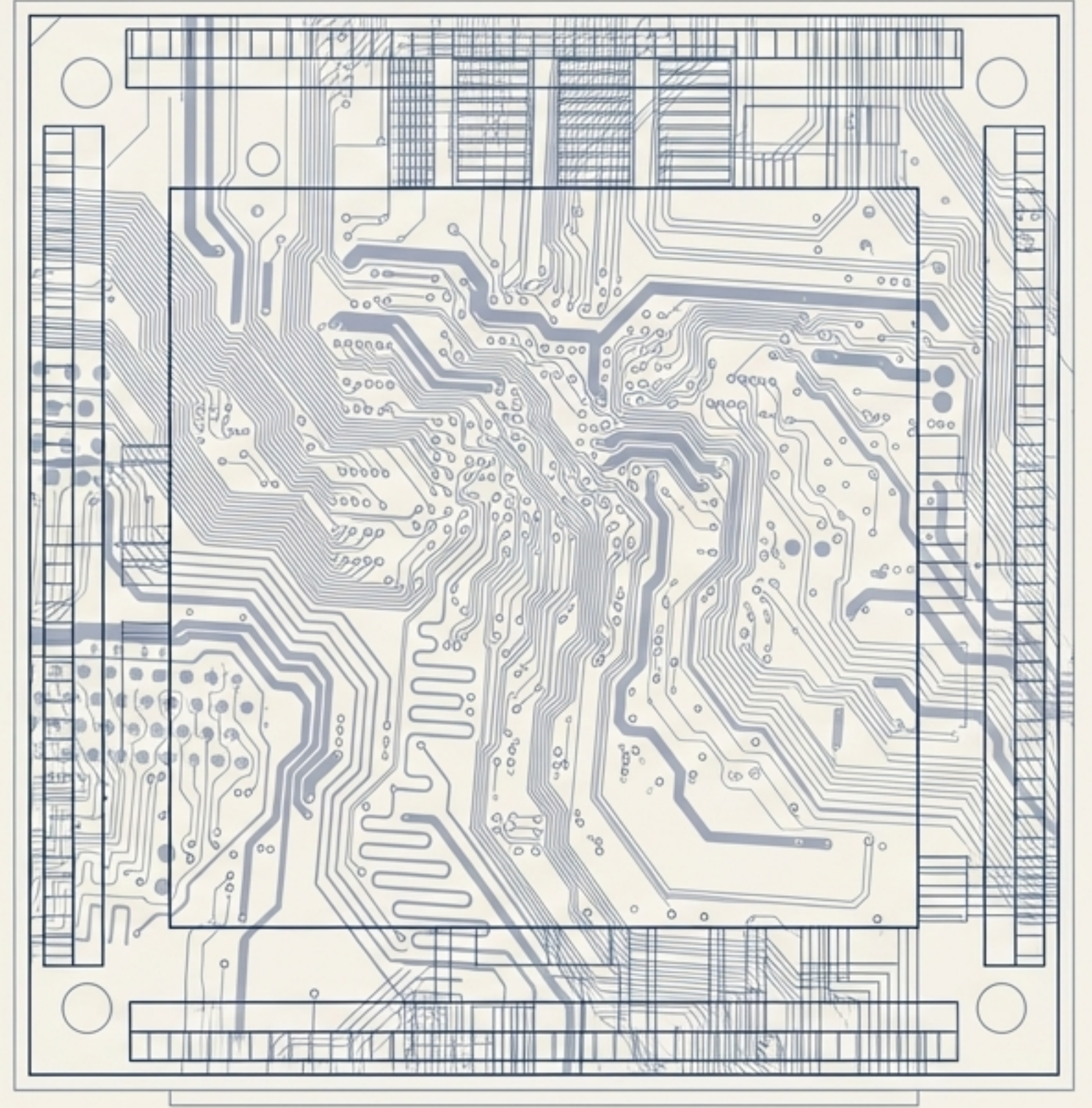


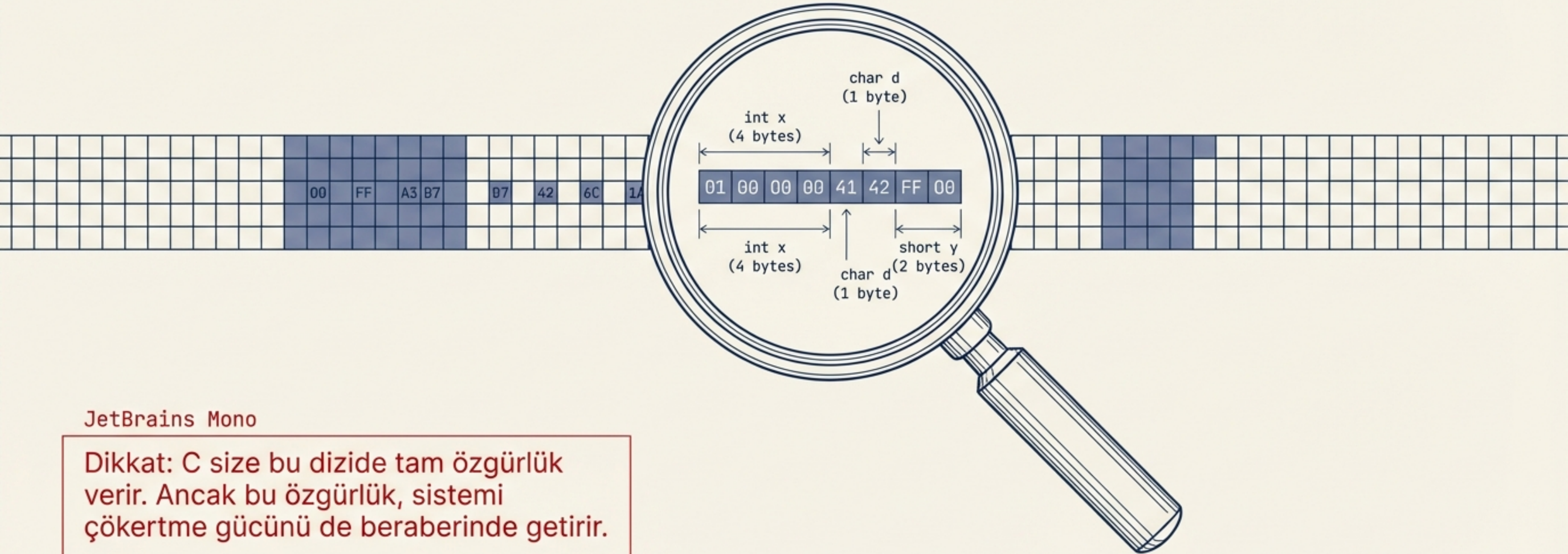
Sistem Programlama: Belleğin Derinliklerine İnmek

C Derleyicisinin ve Donanımın
Görünmez Kuralları



Bellek Dev Bir Bayt Dizisidir

C programlama dilinde bellek, sonu gelmeyen bir hücreler şeridi gibidir. İstedığınız bölgeye tamsayılar (int), karakterler (char) veya yapısal veriler (struct) yerleştirebilirsiniz.



İşletim Sistemi Sınırları: Segmentasyon İhlalleri

```
○ ○ ○
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char *s;

    s = NULL;

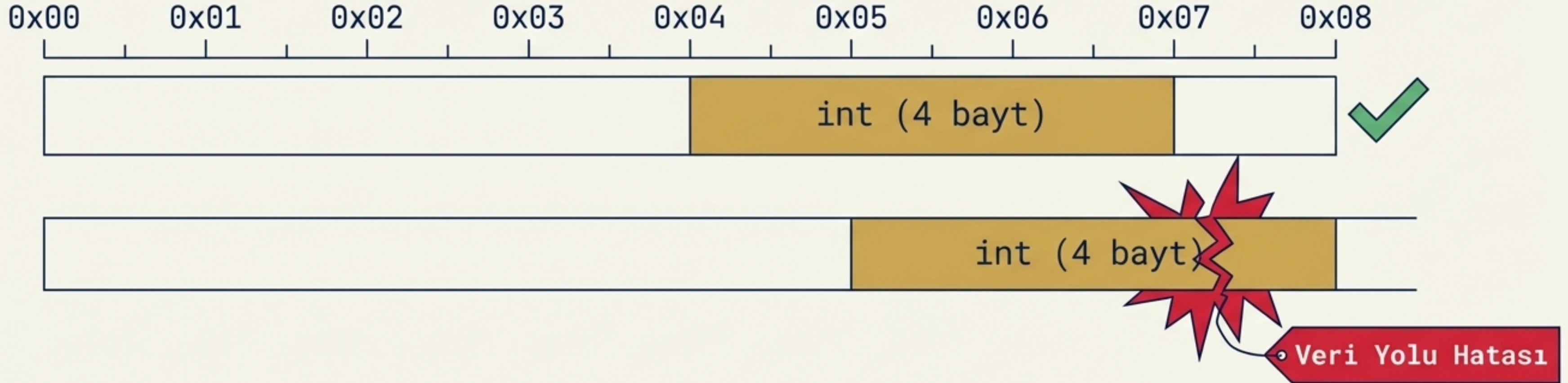
    /* Seg fault here */
    printf("%d\n", s[0]);
    return 0;
}
abduallah@abduallah-VirtualBox:~/sist_prog/cs360-lecture-notes/CStuff-2/bin$ ./pa
Parçalama arızası (çekirdek döküldü)
abduallah@abduallah-VirtualBox:~/sist_prog/cs360-lecture-notes/CStuff-2/bin$ █
```



- Belleğin belirli bölümleri işletim sistemi tarafından donanımsal koruma altındadır.
- Başlatılmamış (NULL) bir pointer'a erişmeye çalıştığınızda donanım müdahale eder.
- Sonuç: Segmentasyon İhlali (Segmentation Fault).

Donanım Kuralları: Veri Yolu Hataları (Bus Errors)

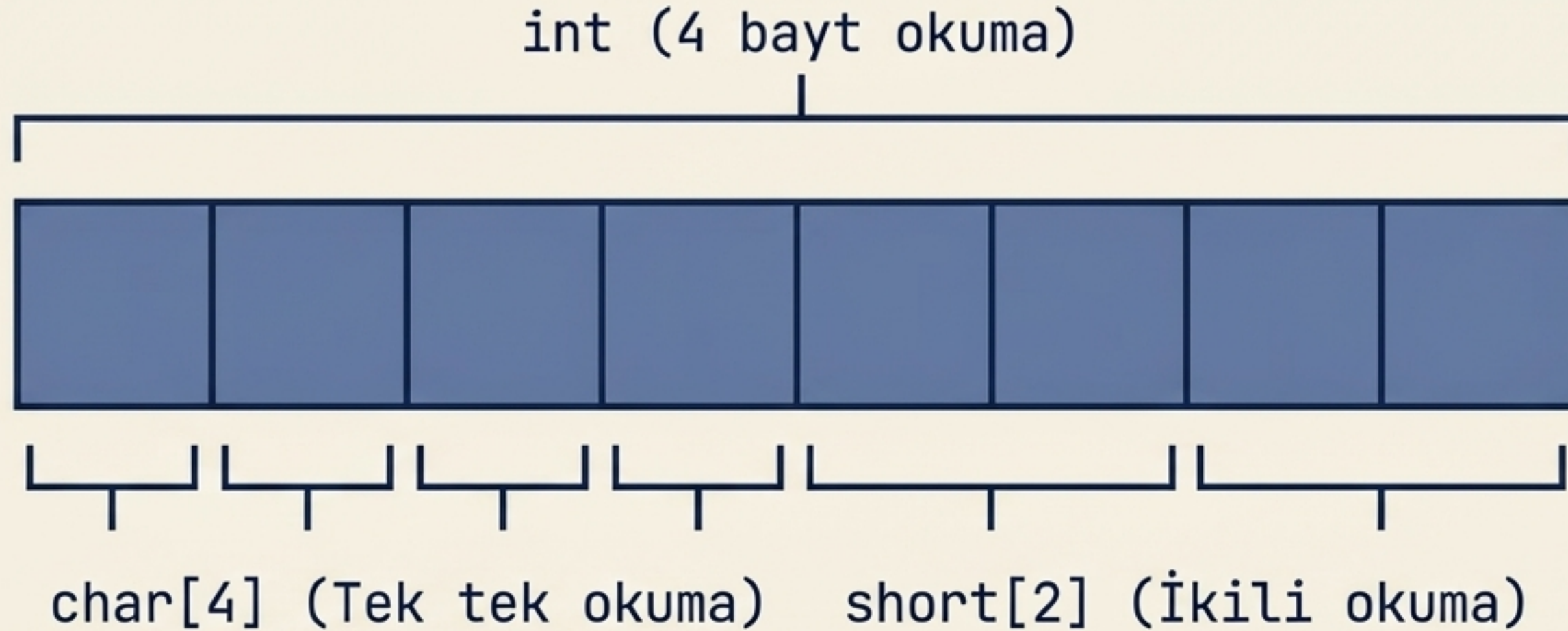
Birçok makinede skaler bir türe eriştiğinizde konum "sıralı" (aligned) olmalıdır.



Kural: Veri türü 4 bayt ise (int), bellekteki konumu mutlaka 4'ün katı olan bir dizinde başlamalıdır. Aksi takdirde CPU veri yolu hatası verir.

C'nin Tehlikeli Esnekliği

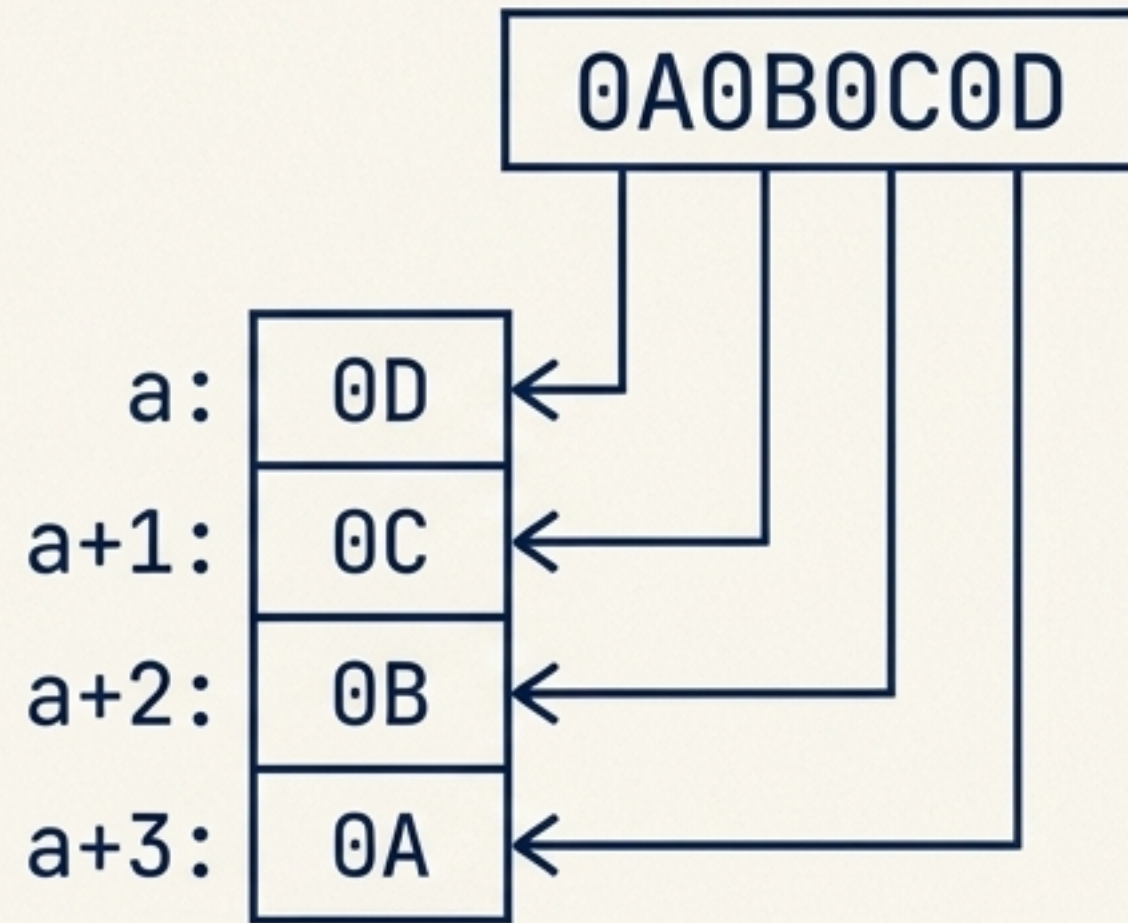
Elinizdeki bir bayt bölgesini istediğiniz herhangi bir tür olarak okuyabilirsiniz.



Sistem programlamada makinenin bellekte ne yaptığını anlamak için bu X-Ray görüşüne ihtiyacımız vardır.

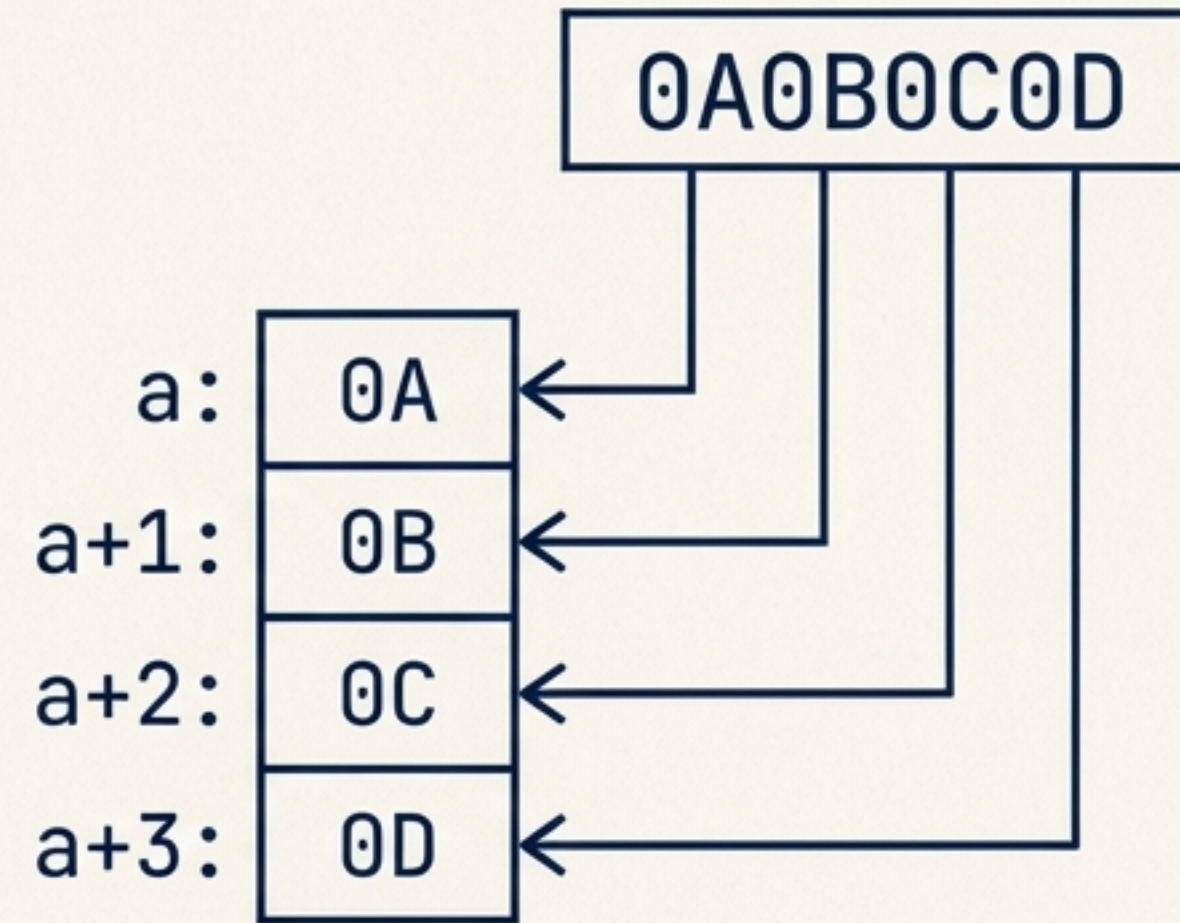
Endianness: Baytların Mimari Yönü

Little-endian



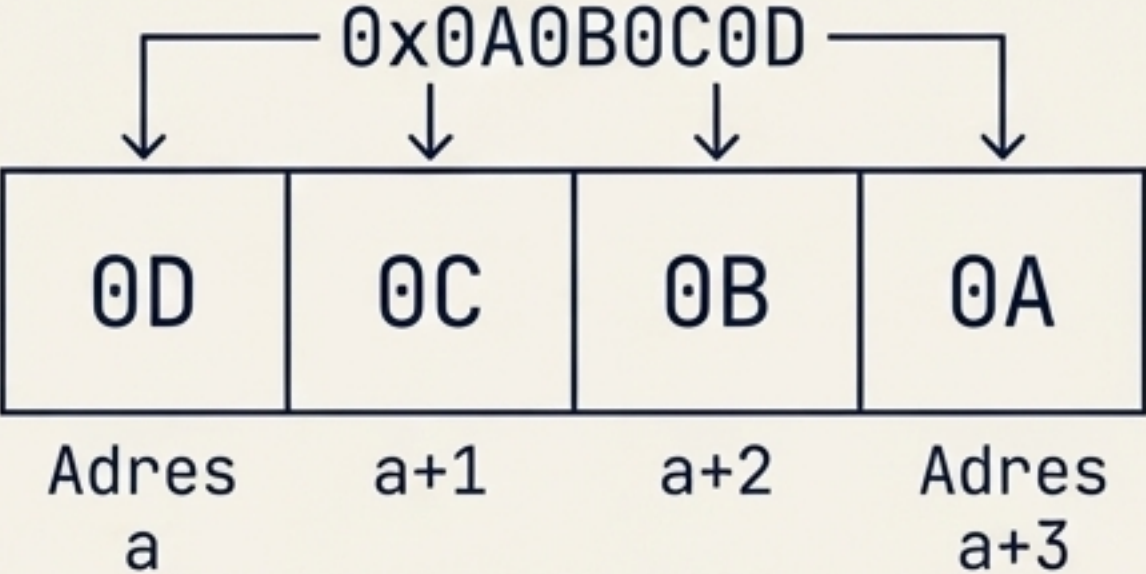
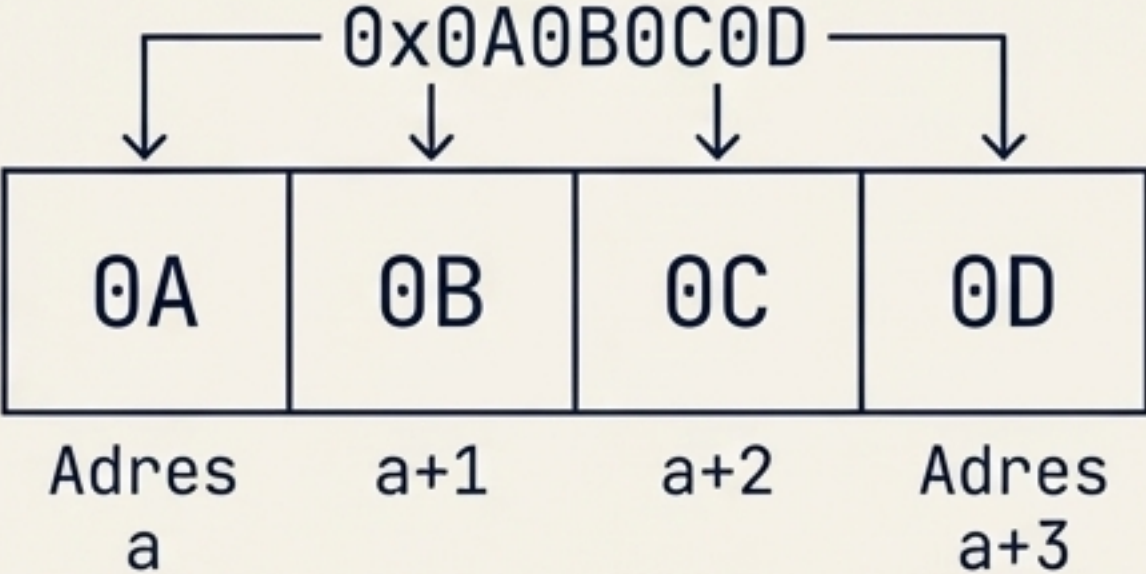
Little-Endian: Verinin en düşük değerlikli (least significant) baytı bellekte ilk sıraya yazılır.

Big-endian



Big-Endian: Verinin en yüksek değerlikli (most significant) baytı bellekte ilk sıraya yazılır.

Endianness Karşılaştırma Matrisi

	Little-Endian	Big-Endian
Bellek Yerleşimi (0x0A0B0C0D)		
Okuma Yönü	Sağdan sola yazım mantığı.	Soldan sağa yazım mantığı.
Kullanan Mimariler	Intel, AMD, ARM (yaygın mod)	Sun Sparc, Motorola 68K, Java Sanal Makinesi (Java VM)

Belleği Bayt Bayt İncelemek

```
#include <stdio.h>

typedef unsigned long UL;

int main()
{
    unsigned int array[4];
    unsigned int *ip;
    unsigned char *cp;
    unsigned short *sp;
    int i;

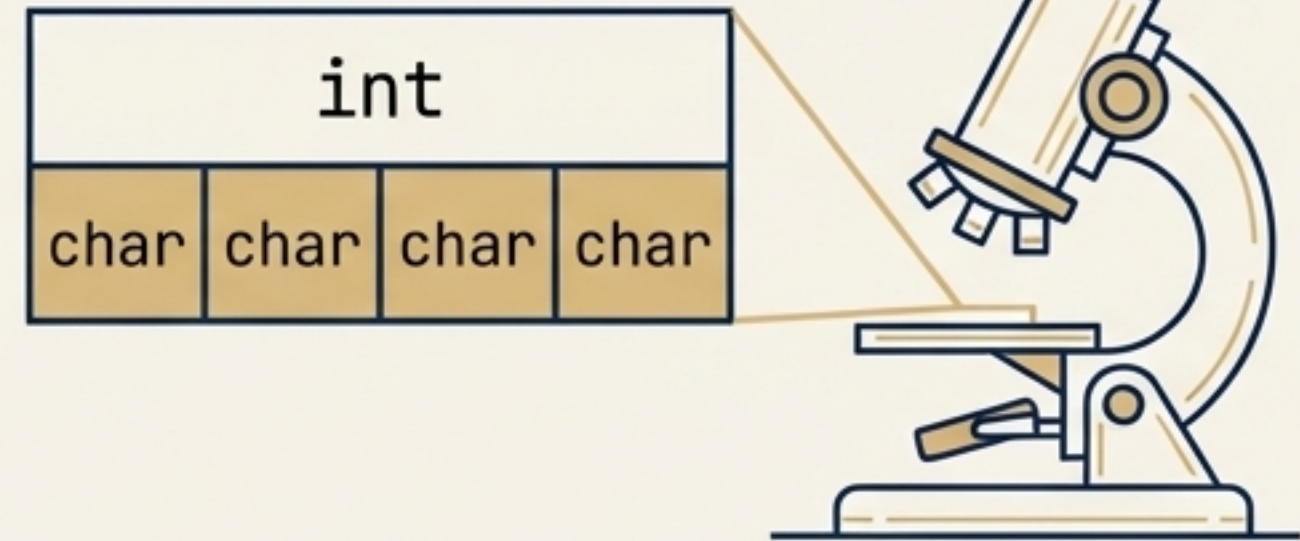
    array[0] = 0x12345678;
    array[1] = 0x9abcdef0;
    array[2] = 0x13579bdf;
    array[3] = 0x2468ace0;

    cp = (unsigned char *) array;

    // ... rest of the code
}
```

Tür Dönüşümü (Type Casting)

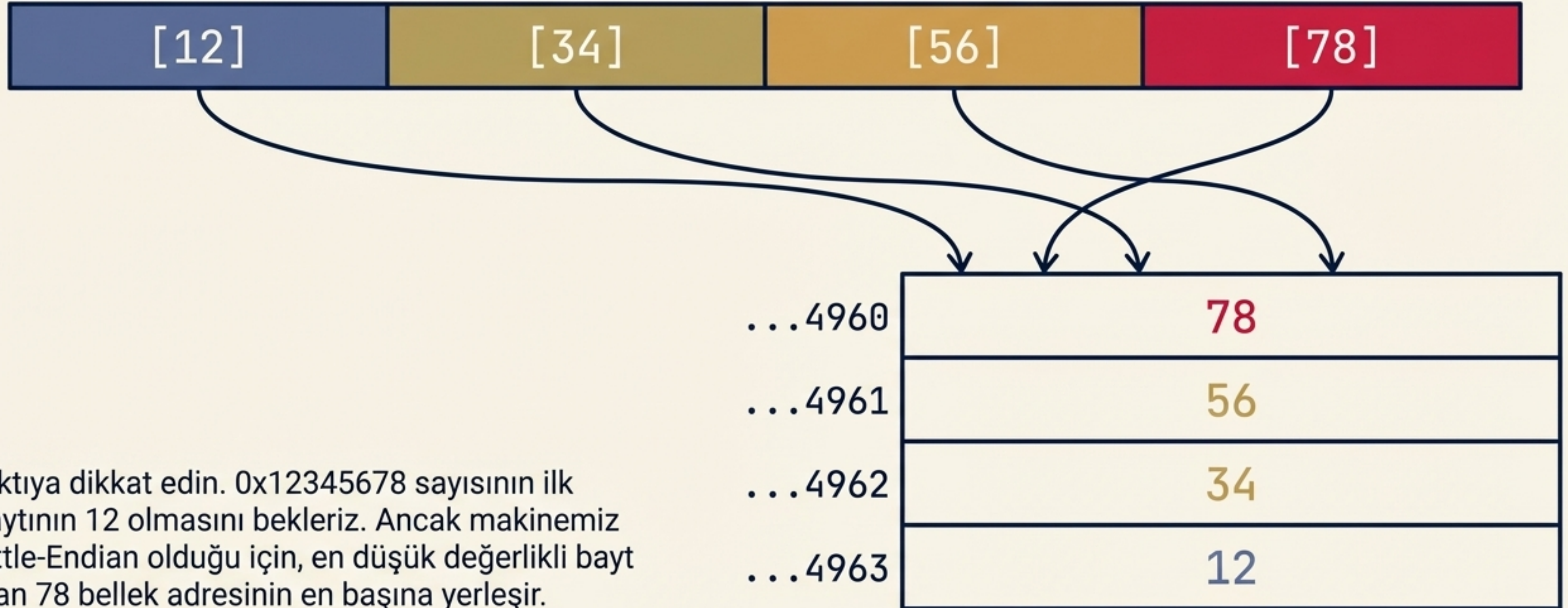
Tam sayılardan oluşan bir diziyi tek tek baytlar halinde görebilmek için, dizinin adresini bir unsigned char * (işaretsiz karakter işaretçisi) tipine zorlarız.



Bu işlem bize belleğe mikroskopta bakma imkanı verir.

Görünmez Çeviri: Bellek Dökümü

0x12345678 (Kavramsal Değer)

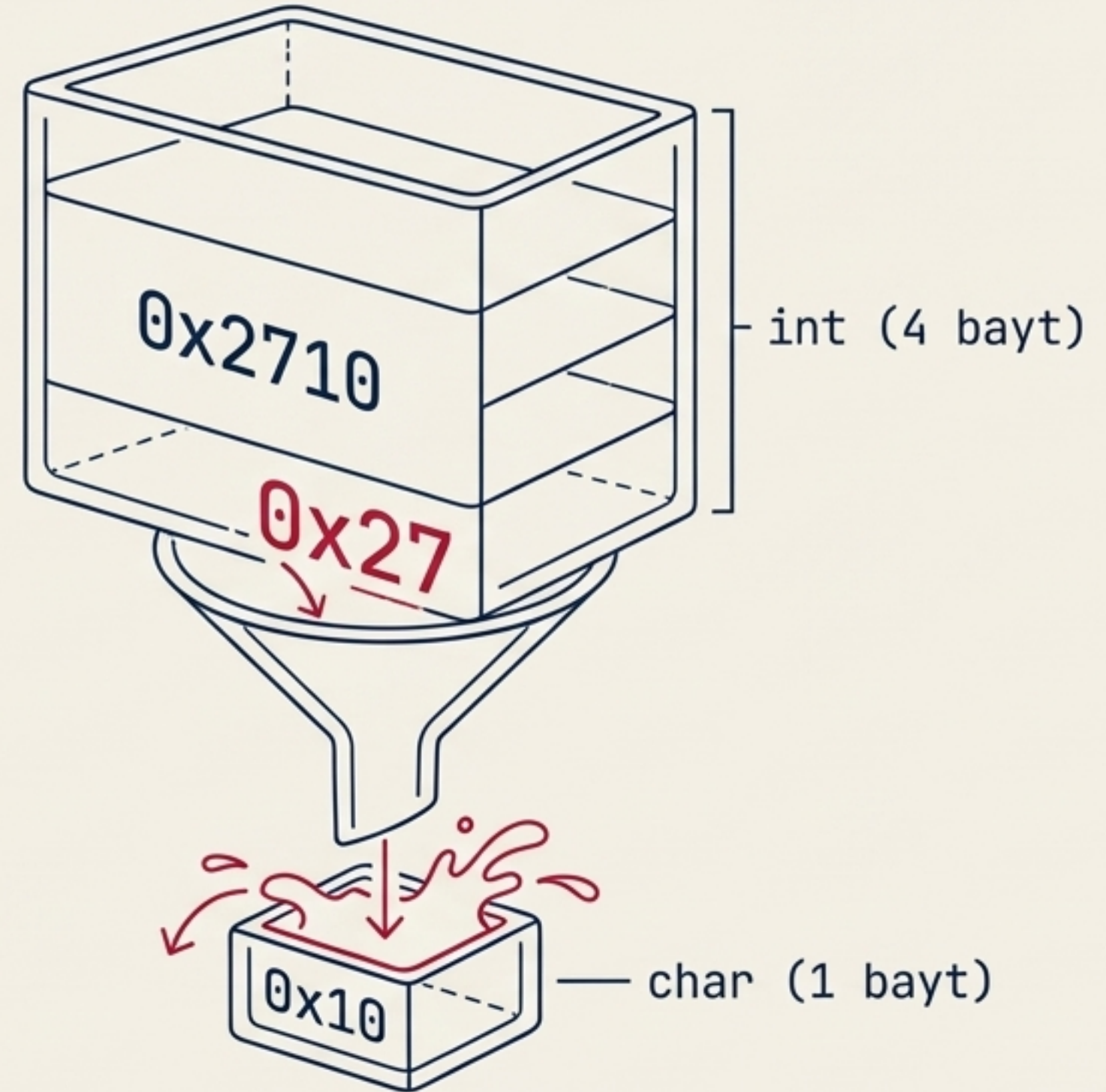


Çıktıya dikkat edin. 0x12345678 sayısının ilk baytının 12 olmasını bekleriz. Ancak makinemiz Little-Endian olduğu için, en düşük değerlikli bayt olan 78 bellek adresinin en başına yerleşir.

Tür Dönüşümü Kayıpları (Truncation)

```
int i = 10000;  
char c = i;
```

- Büyük tiplerden küçük tiplere veri aktarırken bilgi kaybedersiniz.
- 10000 sayısı bir bayta sığmaz. Yalnızca en düşük bayt olan 0x10 (16) saklanır. `char c = 10000;` dediğinizde c'nin değeri artık 16'dır.



Struct Hizalama (Alignment) Kuralları

Kural 1: malloc() Davranışı



malloc() fonksiyonu, ayrılan belleği hangi veri türünün kullanacağını bilmediği için güvende olmak adına her zaman **8'in katları olan bellek adresleri** döndürür.

0x...00
0x...08
0x...10
0x...18
0x...20

Kural 2: Derleyici (Compiler) Dolguları



Derleyici, değişkenleri bellekte sıralı olacak şekilde düzenler. İşaretçilerin kuralını çiğnememek için yapıların arasına **dolgu** (padding) koyarak boyutu beklediğinizden daha büyük hale getirir.



Dolgu Baytları (Padding): Neden 5 Değil 8 Bayt?



char b (1 bayt) ve int i (4 bayt) kağıt üzerinde 5 bayt yer kaplar. Ancak int işaretçisinin adresi 4'ün katı olmalıdır. Derleyici bu kuralı çiğnememek için araya 3 baytlık "boşa harcanmış" dolgu alanı ekler. Toplam boyut 8 bayt olur.

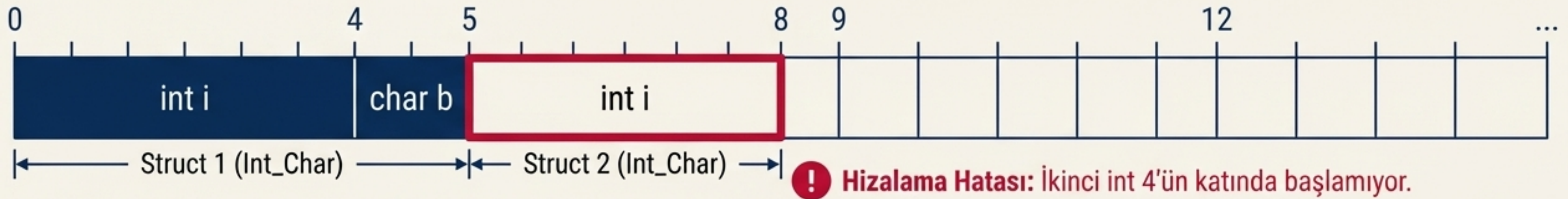
Bellek Optimizasyonu: Değişken Sırası Önemlidir

Yapı (Struct)	Tanımlanan Veri	Gerçek Bellek Boyutu	Kayıp (Doğru)
Char_Int	1 char, 1 int	8 bayt	3 bayt kayıp ⚠️
CCCC_Int	4 char, 1 int	8 bayt	0 bayt kayıp ✅
C_I_C_I	char, int, char, int	16 bayt	6 bayt kayıp ❌
Int_Char	1 int, 1 char	8 bayt	3 bayt kayıp ⚠️

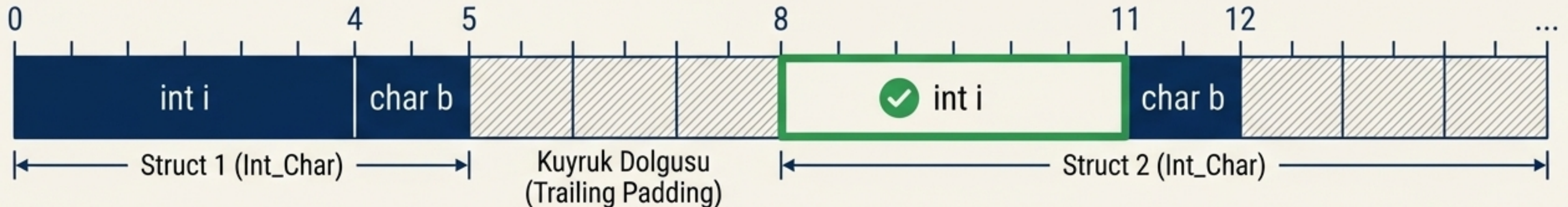
Önemli Çıkarım: Aynı verileri tutan yapılar, sadece tanımlanma sırasına bağlı olarak bellekte %100 daha fazla yer kaplayabilir! Değişkenleri büyükten küçüğe sıralamak en iyi pratiktir.

Dizi Etkisi (The Array Effect)

Hizalama Sorunu (The Problem)



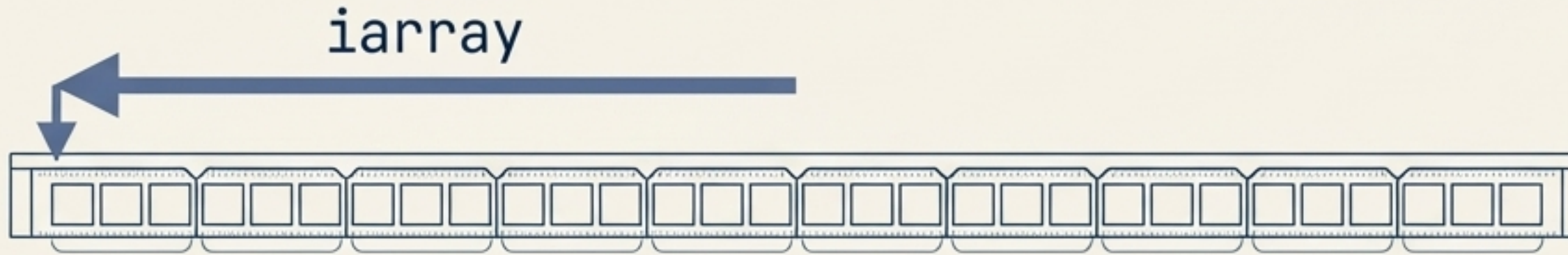
Derleyicinin Çözümü (The Compiler's Solution)



Int_Char yapısı kendi içinde hizalama sorunu yaşamasa da boyutu 5 değil 8 bayttır. Neden? Sadece tek bir struct değil, struct dizileri oluşturduğumuzu düşünün. Derleyici, dizideki bir sonraki elemanın hizalanmasını garanti etmek için ilk struct'ın sonuna dolgu ekler.

Diziler ve İşaretçiler (Arrays and Pointers)

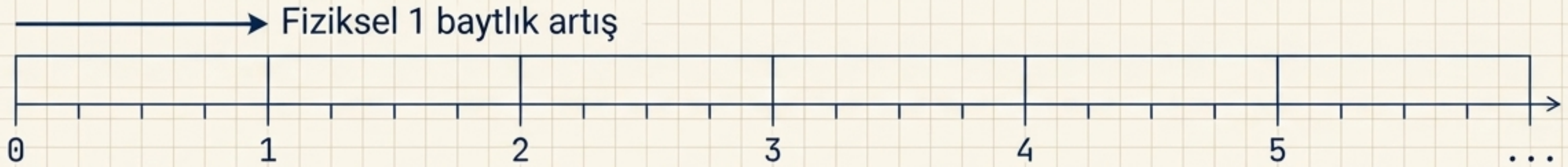
```
int iarray[10];
```



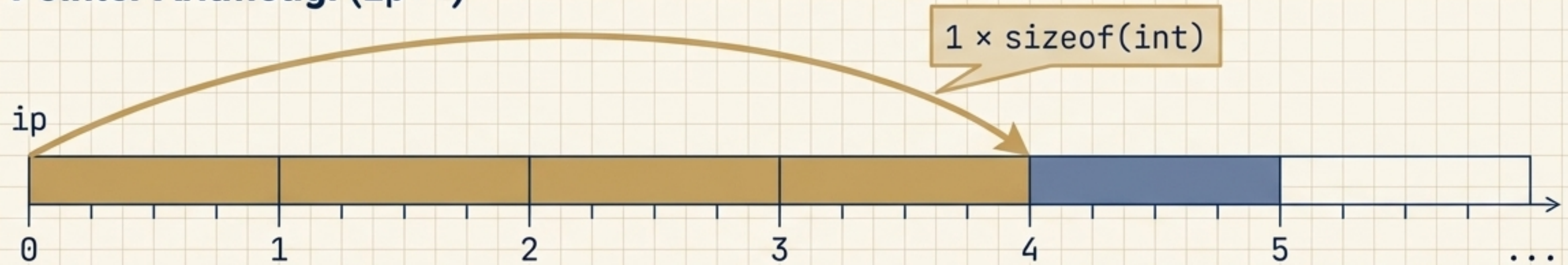
- C'de bir dizinin boyutu hiçbir yerde özel olarak saklanmaz; bunu sizin takip etmeniz gerekir.
- Gerçekte `iarray` değişkeni, dizinin tamamı değil, yalnızca bellekten ayrılan 40 baytlık bloğun ilk elemanına işaret eden bir adrestir.

Pointer Adımları (Pointer Strides)

Normal Matematik (+1)



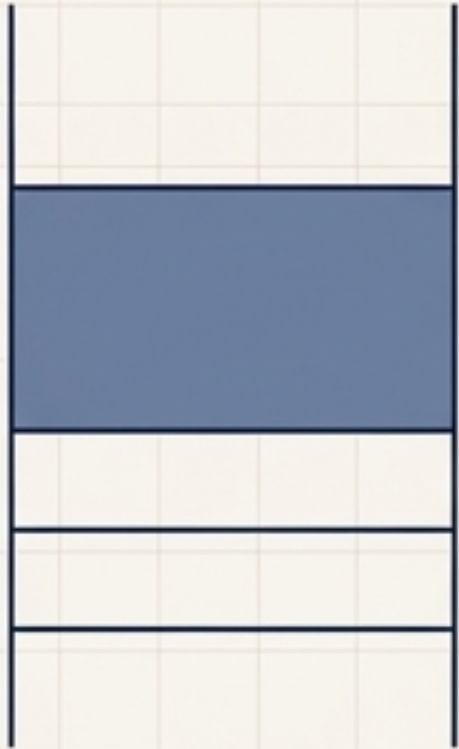
Pointer Aritmetiği (ip++)



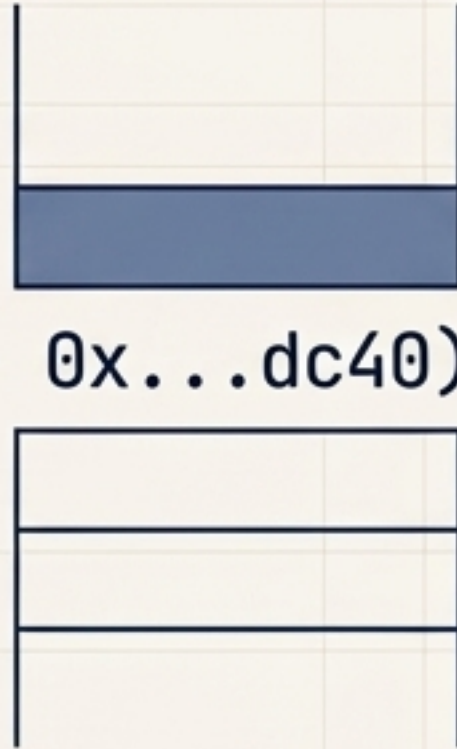
ip bir işaretçi olduğunda, ip++ yapmak fiziksel bellek adresine '1' eklemek demek değildir. Derleyici, adrese gizlice $(1 \times \text{sizeof}(\text{veri_türü}))$ ekler. int pointer'ını 1 artırmak, adresi 4 bayt ileri sıçratır.

Derleyicinin Görünmez Matematiği

iarray = 0x...dc40



ip = 0x...dc44



$(0x...dc44 - 0x...dc40) \rightarrow$ ~~4 bayt fark~~ $\rightarrow (ip - iarray) = 1$

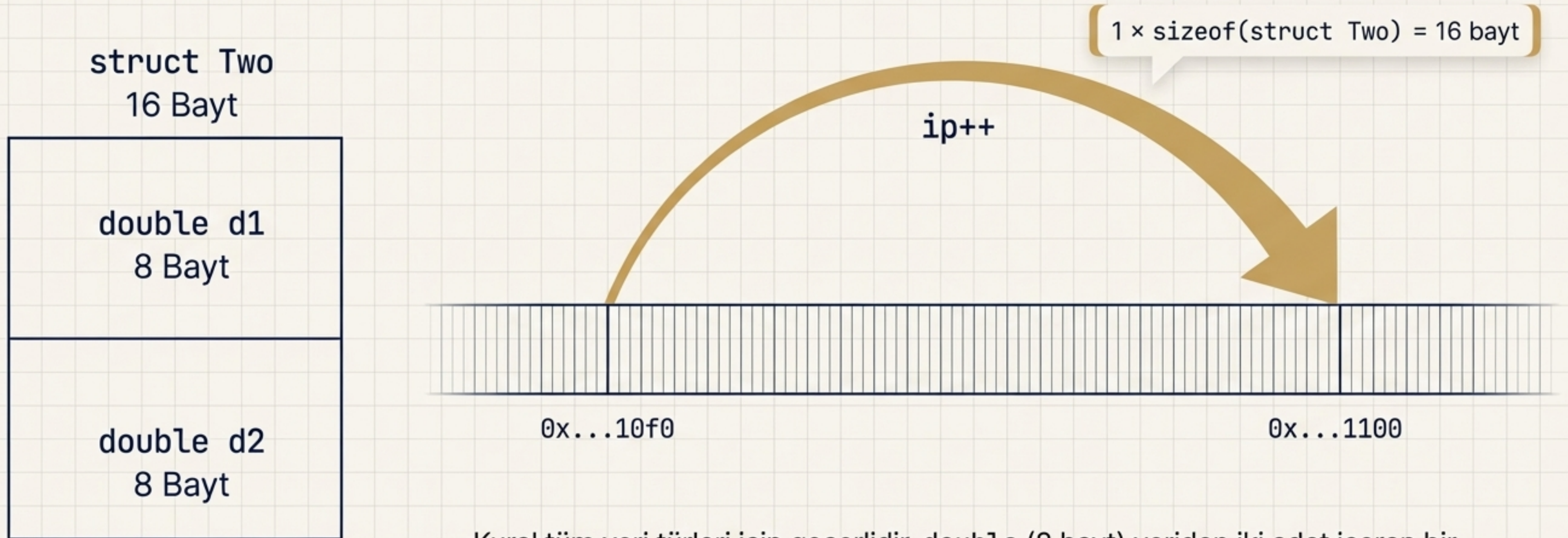
Derleyicinin Gizli İşlemi

Derleyici adres farkını otomatik olarak öge boyutuna böler.
 $(4 \text{ bayt} / \text{sizeof(int)}) = 1 \text{ öge}$.

JetBrains 0x....dc40, iarray - 0x....dc4) = JetBrains Mono.

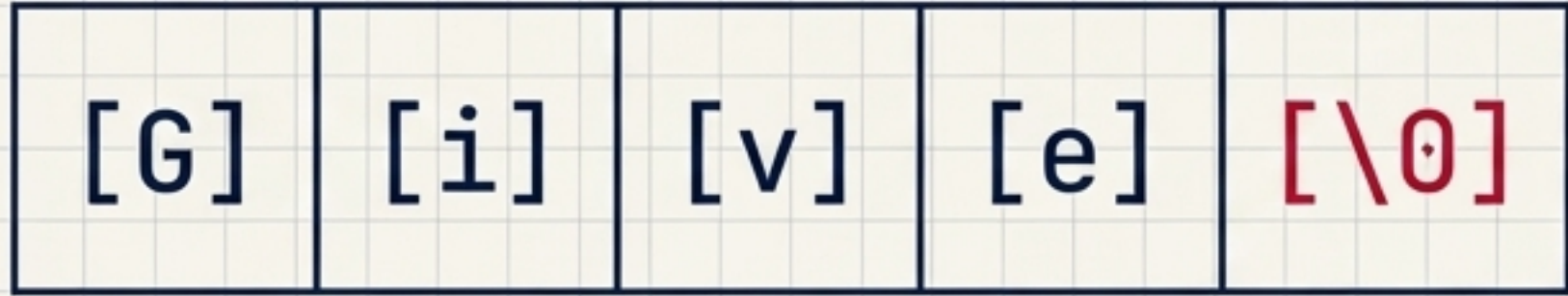
Derleyici işaretçiler arası farkı hesaplarken fiziksel bayt farkını değil, aradaki öge sayısını hesaplar.

Struct'larda Pointer Aritmetiği



Kural tüm veri türleri için geçerlidir. double (8 bayt) veriden iki adet içeren bir struct yapısının boyutu 16 bayttır. Bu struct'ı gösteren bir pointer'ı (ip++) artırdığınızda, derleyici adresi her adımda tam 16 bayt ileri atlatır.

C'de String'lerin Doğası



Null Sonlandırıcı
(Null Terminator)

C'de string kavramı aslında karakterlerden oluşan basit bir dizidir ve sonlandığını belirten sihirli bir işarete dayanır: Null Baytı (\0).

Sınır Tanımazlık: `strcpy()` fonksiyonu bu null baytını görene kadar körü körüne kopyalama yapar. Hedef bellekte yeterli yer olup olmadığını kontrol etmez.

Yığın Taşması (Buffer Overflow / Stack Smashing)

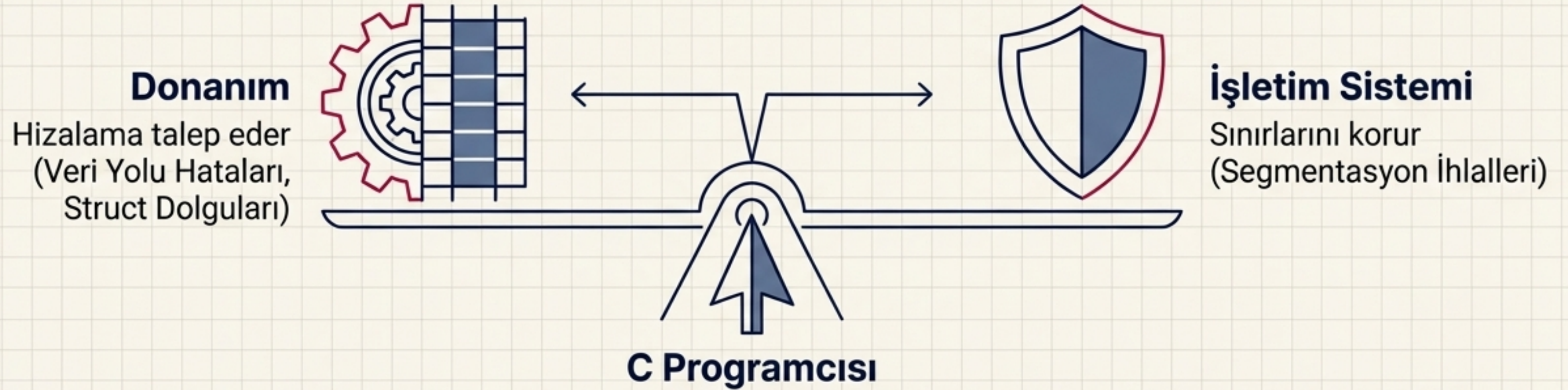
```
strcpy(him, "T.J. Houshmandzadeh");
```

char him[5]



5 karakterlik bir diziye devasa bir isim kopyalamaya çalıştığınızda, strcpy() sınırları aşar ve bellekteki diğer verileri ezer. Modern derleyiciler bu durumu fark edip programı iptal etmeye çalışsa da (stack smashing detected), dikkatli olunmazsa yıkıcı sonuçlar doğurur.

Sentez: Yazılım Özgürlüğü vs. Donanım Kuralları



Pratik Görev (X-Ray Vizyonunuzu Test Edin)

1. Komut satırından 5 kelime alıp `strcat` ile bir `char` dizisine ekleyin ve bellek adreslerini pointer aritmetiği ile bozmadan ekrana yazdırın.
2. Aynı işlemi 5 tamsayı (`int`) değer alarak, onları tek boyutlu bir diziye katarak tekrarlayın.